

Phase 1: Architecture

Sep 24, 2026 · @Asher

Block diagram

Three chips do almost all the work: the MCU runs the control loops, the DRV8316 turns its PWM into motor current, and the encoder tells the MCU where the rotor is.

flowchart LR

```

    BUS["24 V bus in/out<br/>bus connector pins"] --> PROT["Reverse
    polarity<br/>+ TVS + bulk caps"]
    PROT --> DRV["DRV8316<br/>driver + FETs + buck"]
    DRV -->|"3 phases"| MOTOR["Motor<br/>solder pads"]
    DRV -->|"3.3 V"| MCU["STM32G431<br/>MCU"]
    MCU -->|"PWM, SPI, enable"| DRV
    DRV -->|"current sense, nFAULT"| MCU
    ENC["MT6701 encoder<br/>motor side"] -->|"SSI"| MCU
    MCU <-->|"FDCAN"| XCVR["CAN transceiver"]
    XCVR <--> CAN["CAN in/out<br/>+ ENABLE line"]
    CAN -->|"ENABLE"| DRV
    AUX["Aux header<br/>output sensor or switch"] --> MCU
    SENSE["Bus voltage divider<br/>+ NTC"] --> MCU
    SWD["TC2030 SWD pads"] --- MCU
    MCU --> LED["Status LED"]
  
```

Power flows left to right along the top; control and feedback run between the MCU and the chips around it.

- **DRV8316:** three half-bridges, current sensing without shunt resistors, and the buck that makes 3.3 V, all in one 7×5 mm package. That integration is the main reason it fits a 40 mm board ([TI](#)).
- **SPI variant (DRV8316R or DRV8316CR):** lets firmware set the current limit and read fault codes. The hardware-strap variant (DRV8316T) needs resistors for every setting and can't tell you why it faulted.
- **ENABLE line:** gates the driver in hardware as well as telling the MCU, so an E-stop works even if firmware hangs.
- **Encoder link:** SSI (a read-only SPI) gives a full 14-bit angle in one fast read; I2C is too slow for a commutation loop.

Power tree

24 V passes straight through each board to the next; each board taps it through its own protection, feeds the DRV8316 directly, and gets 3.3 V from the DRV8316's built-in buck.

flowchart LR

```

    PSU["24 V 10 A supply"] --> FUSE["5 A inline fuse"]
    FUSE --> IN["24 V in, bus connector"]
    IN --> OUT["24 V out, bus connector<br/>to next board"]
    IN --> RP["P-FET reverse<br/>polarity switch"]
    RP --> TVS["TVS clamp<br/>+ damped bulk cap"]
    TVS --> VM["DRV8316 VM<br/>motor power"]
    VM --> BUCK["DRV8316 buck<br/>3.3 V, 200 mA max"]
    BUCK --> L33["MCU, encoder,<br/>CAN transceiver, LED"]
    TVS --> DIV["Divider to ADC<br/>bus voltage"]
  
```

Each block, and why it's there:

- **Fuse at the supply:** a 5 A blade fuse matches the connector's single +24 V pin and protects the harness from a partial short that stays under the supply's own ~10 A trip.
- **Pass-through first:** the in and out connector pins are joined by a wide copper pour ahead of any protection. Each board protects only itself, so one board's parts never carry the rest of the arm's current.
- **P-FET reverse-polarity switch:** a MOSFET that only turns on when polarity is correct. A diode would also work but drops about 0.5 V and burns about 1 W at 2 A. Once on, the P-FET conducts both ways, so braking energy can flow back out to the other boards instead of piling up on this one. A Zener protects its gate, since 24 V exceeds the usual ± 20 V gate rating.
- **TVS clamp:** a diode that does nothing below its standoff voltage (about 26 V) and clamps spikes above it. It keeps spikes under the DRV8316's 40 V absolute max ([TI](#)).
- **Damped bulk capacitor:** plugging 24 V into a board with only ceramic capacitors can ring to nearly double, about 48 V, which would kill the driver. A polymer or electrolytic cap has enough internal resistance to damp that ring. Small ceramics still sit right at the VM pin.
- **Regen:** these high-resistance motors burn most braking energy in their windings and the gearbox friction. The TVS, the driver's own overvoltage protection and a firmware bus-voltage cutoff cover the rest.
- **3.3 V from the DRV8316 buck:** it saves a regulator chip on a crowded board. Its 200 mA covers the estimated load below with margin. It needs one small inductor.

3.3 V load	Estimated current
STM32G431 at full speed	30 to 40 mA
MT6701 encoder	10 to 14 mA
CAN transceiver while transmitting	up to about 60 mA
Status LED, NTC, pull-ups	about 5 mA
Total	about 120 mA of 200 mA

Estimates only; exact figures come from each datasheet in Phase 2.

Two rules keep the MCU alive:

1. **E-stop uses DRVOFF, never sleep.** Pulling DRVOFF high turns off all six FETs while the buck, the driver's 3.3 V LDO and SPI stay running ([datasheet 8.4.2](#)). So during an E-stop the MCU keeps reporting over CAN.
2. **nSLEEP stays tied awake.** Sleep mode shuts down the buck, which would cut the MCU's own power.

CAN bus and harness

Each cable end uses one 6-pin Molex Micro-Fit 3.0 connector carrying both power and CAN, so every segment is a single cable. Each board has two, one in and one out.

Why this connector

- **Rating:** 3.0 mm pitch, dual row, latching, 18 to 30 AWG terminals, and about 5 A per pin with 20 AWG wire ([MiniProto, from Molex data](#)). It's rated for about 30 mating cycles, fine for a build but not for daily unplugging.
- **Height:** it will be the tallest part on the board, about 17.6 mm mated for the vertical version chosen in Phase 2.
- **Ribbon ruled out:** 28 AWG ribbon and most IDC sockets are rated about 1 A per conductor, flat cable handles twist badly, and mid-cable crimps lock in board spacing ([Mosaic, citing 3M and Belden data](#)). That also makes the 1.27 vs 2.54 mm pitch question moot.

- **Flat ribbon through rotating joints, honestly:** it can work, but only as a properly designed clock spring (a flat spiral that winds and unwinds). A simple loop of ribbon at a joint mostly twists. Round high-flex wire with a generous service loop is the easy, proven route.

Pinout

Viewed as a grid, with pin numbers assigned from the housing drawing in Phase 3:

	Column 1	Column 2	Column 3
Row A	+24 V	GND	CAN_H
Row B	ENABLE	GND	CAN_L

The two GND pins in the middle column separate the power side from the CAN side, so 24 V never sits next to a CAN pin. Both GNDs carry motor return current in parallel.

Wire per segment: 20 AWG silicone for +24 V and both GNDs; 22 AWG for ENABLE, CAN_H and CAN_L, with CAN_H and CAN_L twisted together. Sleeve all six as one cable.

What sharing a connector changes:

- **CAN transceiver:** switch to TI's TCAN3413 (see Phase 2), whose bus pins survive 58 V instead of ± 14 V ([TI](#)). That covers a crushed or miswired cable. LCSC stock checked in Phase 2.
- **ENABLE input:** each board's ENABLE input must survive 24 V, since it sits next to the +24 V pin.
- **Fuse:** drops from 7.5 A to 5 A to match the single +24 V pin.

Why 5 A is generous: bus current is much lower than motor current at low speed. A 4015 holding at its 1.4 A rated current burns about 10 W in its windings, which is only about 0.44 A from the 24 V bus. All six joints holding a pose draw roughly 1.5 A total.

Voltage drop: 20 AWG copper is roughly $0.033 \Omega/\text{m}$. At 5 A over 1.5 m out and back, that's about 0.5 V, which the motors won't notice.

Topology and termination

flowchart LR

```

    PC["PC or SBC<br/>ROS2"] -->| "USB" | ADPT["USB-CAN adapter<br/>120 Ω on"]
    ADPT --> BASE["Base junction<br/>24 V, CAN, ENABLE"]
    ESTOP["E-stop button<br/>normally closed, 5 V"] --> BASE
    PSU["24 V supply<br/>+ 5 A fuse"] --> BASE
    BASE --> J1["J1 base"] --> J2["J2 shoulder"] --> J3["J3 elbow"]
    J3 --> J4["J4 wrist 1"] --> J5["J5 wrist 2"] --> J6["J6 wrist 3<br/>120 Ω on"]
  
```

One 6-conductor cable runs from the base junction to J1, then board to board up to J6.

- **Termination:** each board carries a split terminator, two 60 Ω resistors in series across CAN_H and CAN_L with a small capacitor from their midpoint to GND. The capacitor drains motor PWM noise to ground. Two solder jumpers, one per line, connect it; close them only on J6. The adapter terminates the other end.
- **Stubs:** the pass-through connectors keep each board's branch to a few centimeters of trace, far below the roughly 0.3 m limit at 1 Mbit/s.
- **E-stop:** a normally closed button at the base feeds 5 V from a small 24-to-5 V buck module onto the ENABLE pin. On each board, 5 V present turns on a small NPN transistor that holds DRVOFF low (driver enabled). Press the button or break the wire, and a resistor pulls DRVOFF high, so all six FETs switch off in hardware.
- **Spare lines:** none on 6 pins. An 8-pin Micro-Fit would add two, at one more pin column of width.

Encoder mounting geometry

The MT6701 sits dead center on the board's motor-facing side, about 1 mm from the magnet, and must stay within 0.3 mm of the motor axis.

Parameter	MT6701 datasheet	This design
Magnet	Ø6 × 2.5 mm diametric disc recommended	Ø10 × Ø5.5 × 2.5 mm diametric ring, factory-fitted
Field at the IC	200 to 1,000 gauss	Unknown for the ring; the vendor's own MT6701 board works at about 1 mm

Parameter	MT6701 datasheet	This design
Air gap, magnet to IC top	0.5 to 2.0 mm, 1.0 typical	1.0 mm target
Off-axis misalignment	0.3 mm max	Budget below
Angle accuracy	$\pm 1.0^\circ$ typical, $\pm 1.5^\circ$ worst case	Improved by firmware calibration
Supply current	10 mA typical, 14 mA max	Fits the 3.3 V budget

Source: [MT6701 datasheet Rev 1.5](#), sections 4 and 5.

Height stack

Height above stator face	4015	2804
Magnet face (from drawings)	about 1.0 mm	about 4.5 mm
Air gap	1.0 mm	1.0 mm
MT6701 SOP-8 body	1.75 mm max	1.75 mm max
PCB underside = spacer length	about 3.75 mm	about 7.25 mm

The spacer is the only part that differs between motors. Print spacers in 0.25 mm steps and keep the one that gives a "normal" field reading, since the magnet heights are drawing values.

Staying on axis

Error source	Typical size
M2.5 screw rattling in a 2.7 mm hole	up to 0.1 mm
Hole position from the PCB fab	about 0.1 mm
IC placement by the assembler	about 0.05 to 0.1 mm
Worst-case total	about 0.3 mm, right at the limit

Fab and assembly figures are typical estimates; I'll confirm JLCPCB's numbers in Phase 5. Tightening the mounting holes to 2.6 mm takes most of the screw rattle out.

Why it matters: the motor sees angle error multiplied by its pole pairs. A 1.5° error on the 11-pole-pair 4015 is 16.5° electrical, costing about 4% of torque plus some ripple. After a 30:1 gearbox it's only 0.05° at the joint, so it affects smoothness, not accuracy. SimpleFOC's sensor calibration removes most of it.

Built-in health check

Every SSI read returns the 14-bit angle plus status bits for "field too strong", "field too weak" and "loss of track", and a CRC. Firmware can flag a bad gap or a corrupted read live, and bring-up uses the same bits to pick the spacer.

Keep-outs and one pitfall

- **Stray fields:** 5 A in a phase trace 5 mm away adds about 2 gauss, 1% of the minimum magnet field, worth up to about half a degree. Keep phase traces and the buck inductor well away from the sensor, and run each phase trace close to its return so their fields cancel.
- **Don't program the EEPROM:** the MT6701 only programs its EEPROM above 4.5 V. We run it at 3.3 V and handle the zero offset in firmware instead.

Decisions to confirm before Phase 2

Confirmed on Sep 24; Phase 2 builds on these.

1. **Harness:** one 6-pin Micro-Fit 3.0 connector per cable end, carrying 24 V, CAN and ENABLE in a single cable. Chosen over the two-cable layout so each segment has one plug.
2. **CAN connector size:** no longer applies, since CAN now shares the 6-pin connector.
3. **E-stop 5 V source:** a small 24-to-5 V buck module at the base, so the arm powers up from one supply.